seed42 Audio Module: Task Assignments

Phase 1 build. Target: by 30 August, take a few songs played one after another and, every 60 seconds, output a classification we can later turn into a prompt. This document says what is already done, the shared shapes everyone builds to, and who does what.

Read AGENTS.md in the repo root before writing any code. It has the hard rules (causal only, Phase 1 only, no em dashes, branch and pull request workflow). Every task below assumes you have read it.

What we already have

- Repo created, public, with branch protection: no direct pushes to main, every change goes through a pull request with one approval. Jayden and Hatim both review and approve pull requests before they merge.
- Project skeleton scaffolded: all folders and stub files, .gitignore, README.md, requirements.txt.
- Eight feature branches created, one per task below.
- AGENTS.md coding guidelines (being merged).
- The causal streaming loop io/stream.py (drafted, being merged). This is the interface everything else reads from.
- The seed42 Control API spec from the sponsor, so we now know the real output contract (REST calls to POST /api/prompts, hot-swap fields only). This mainly affects the output task and is documented separately.

So the interface is fixed and the work below can start in parallel now.

The two shapes everyone builds to

Do not change these without telling Jayden, because other people's code depends on them.

1. A segment, from AudioStream.segments(), yields:

- timestamp: a float, seconds, the trailing edge of the window
- samples: a 1D numpy array of the last window seconds of audio at sample rate sr

2. Every feature function has the same simple signature:

```
def extract(samples, sr) -> dict
```

It takes one segment's samples and the sample rate, and returns a small dict of numbers. No file reading, no state, no future audio. Just numbers out.

3. The shared MusicState (Jayden owns this) is one object per window that collects all the feature outputs, with fields roughly like: timestamp, tempo, energy, brightness, flatness, key, mode, valence, arousal. The pipeline fills it each window and the output step reads it.

Who does what

Each person: work only on your branch, read AGENTS.md, keep it simple, no em dashes, then open a pull request into main for Hatim to review.

Jayden (Ngoc Nhat Pham): the interface and the loop

- Branches: feature/stream-loop, feature/music-state
- Files: src/seed42_audio/io/stream.py, src/seed42_audio/state/music_state.py
- Task: finish and merge the streaming loop, and define the MusicState object (a simple dataclass with the fields above). Later, wire the pipeline loop that runs every 60 seconds.
- Priority: P1. This unblocks everyone.

Hatim Hussaini: rhythm, plus quality

- Branch: feature/rhythm-onsets
- File: src/seed42_audio/features/rhythm.py
- Task: from one segment, extract tempo (BPM), onset positions and beats using librosa (librosa.onset, librosa.beat.beat_track). Return them in the extract(samples, sr) dict.
- Also, as Quality Manager: review each pull request before it merges (together with Jayden), and run the em-dash check grep -rn "—" src/ scripts/ tests/.
- Priority: P1. Level: medium to high (tempo is the fiddliest feature).

Tatiana Denise Bandong: spectral features

- Branch: feature/spectral-features
- File: src/seed42_audio/features/spectral.py
- Task: from one segment, compute spectral centroid (brightness), RMS energy (loudness), spectral flatness (noisy vs tonal), and bass energy (low-frequency share). All are one or two lines each in librosa. Return them in the extract(samples, sr) dict.
- Priority: P1. Level: low to medium. Good first task.

Charminkumar Patel: key and chroma

- Branch: feature/chroma-key
- File: src/seed42_audio/features/chroma.py
- Task: from one segment, compute the chroma vector (librosa.feature.chroma_cqt), then estimate the key and whether it is major or minor (correlate the chroma against major and minor templates and keep the stronger). Return them in the extract(samples, sr) dict.
- Priority: P1. Level: low to medium.

Philopateer Sedrak: mood classification

- Branch: feature/mood-classify
- File: src/seed42_audio/mood/classify.py
- Task: from one segment, produce valence and arousal (and optionally genre and mood) using Essentia's pretrained models. Same extract(samples, sr) style output.
- Note: plain essentia will not install on Windows. Use essentia-tensorflow, or run this on a Mac or Linux machine, or through WSL. If Essentia blocks you, ship a temporary fallback that estimates valence and arousal from brightness, energy and mode, so the pipeline still produces a classification for the 30 August demo. Flag this to Jayden if it happens.
- Priority: P1. Level: medium to high (mainly because of the Essentia setup).

Muhammad Danial Sarfraz: output to seed42 and API docs

- Branch: feature/output-writer
- Files: src/seed42_audio/output/writer.py, docs/seed42_api.md
- Task: two parts.
    1. Write docs/seed42_api.md, a short summary of the sponsor's Control API (the endpoint, the hot-swap vs reload field list, and our label to field mapping), so the whole team builds to one contract.
    2. Start writer.py, the client that sends POST /api/prompts with a partial body. Hard rule: only ever send hot-swap fields (prompt, seed, t_index_list, the three ControlNet conditioning_scale values, ip_adapter). Never send a reload field (model, width, height, ControlNet models), because that causes a 30 second blackout. For now it can just print the request it would send, since we do not have a live stream_id yet.
- Priority: P2. Not needed for the 30 August classification demo, but good to build now that we have the spec. As liaison, also confirm with the sponsor how we get a stream_id and a test stream.

How we work (short version)

1. git checkout your-branch before you start. Never commit on main.
2. Read AGENTS.md. Keep changes small and simple. No em dashes.
3. git add, git commit -m "clear message", git push.
4. Open a pull request into main on GitHub. Ask Jayden or Hatim to review.
5. Once approved, it merges. Pull main before starting your next piece.

A separate step-by-step GitHub guide (clone, branch, commit, push, pull request) is provided for anyone who has not used GitHub much. Ask Jayden if you are stuck at any step.